

GaussNav: Gaussian Splatting for Visual Navigation

Xiaohan Lei , Min Wang , Wengang Zhou , *Senior Member, IEEE*, and Houqiang Li , *Fellow, IEEE*

Abstract—In embodied vision, Instance ImageGoal Navigation (IIN) requires an agent to locate a specific object depicted in a goal image within an unexplored environment. The primary challenge of IIN arises from the need to recognize the target object across varying viewpoints while ignoring potential distractors. Existing map-based navigation methods typically use Bird’s Eye View (BEV) maps, which lack detailed texture representation of a scene. Consequently, while BEV maps are effective for semantic-level visual navigation, they are struggling for instance-level tasks. To this end, we propose a new framework for IIN, Gaussian Splatting for Visual Navigation (GaussNav), which constructs a novel map representation based on 3D Gaussian Splatting (3DGS). The GaussNav framework enables the agent to memorize both the geometry and semantic information of the scene, as well as retain the textural features of objects. By matching renderings of similar objects with the target, the agent can accurately identify, ground, and navigate to the specified object. Our GaussNav framework demonstrates a significant performance improvement, with Success weighted by Path Length (SPL) increasing from 0.347 to 0.578 on the challenging Habitat-Matterport 3D (HM3D) dataset.

Index Terms—Embodied visual navigation, 3D Gaussian splatting.

I. INTRODUCTION

EMBODIED visual navigation is an emerging computer vision problem where an agent uses visual sensing to actively interact with the world and perform navigation tasks [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12]. Recent years have witnessed substantial progress in embodied visual navigation, fueled by the availability of large-scale photo-realistic 3D scene datasets [13], [14], [15] and fast simulators for embodied navigation [16], [17], [18]. These advancements have enabled researchers to develop and test navigation algorithms in controlled environments that closely mimic real-world conditions.

In embodied visual navigation, one critical question is “how can we describe the goal target”. One common setting for tasking

Received 29 June 2024; revised 13 December 2024; accepted 29 January 2025. Date of publication 4 February 2025; date of current version 3 April 2025. This work was supported in part by National Natural Science Foundation of China under Grant 62472141, in part by Key Laboratory of Target Cognition and Application Technology under Grant 2023-CXPT-LC-005, in part by the Youth Innovation Promotion Association CAS, in part by the GPU cluster built by MCC Lab of Information Science and Technology Institution, USTC, and in part by the Supercomputing Center of the USTC. Recommended for acceptance by S. Gould. (Corresponding author: Wengang Zhou.)

Xiaohan Lei, Wengang Zhou, and Houqiang Li are with the MoE Key Laboratory of Brain-inspired Intelligent Perception and Cognition, University of Science and Technology of China, Hefei 230027, China (e-mail: leixh@mail.ustc.edu.cn; zhwg@ustc.edu.cn; lihq@ustc.edu.cn).

Min Wang is with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei 230030, China (e-mail: wang-min@iaai.ustc.edu.cn).

The source code is publicly available at the link: <https://github.com/XiaohanLei/GaussNav>.

Digital Object Identifier 10.1109/TPAMI.2025.3538496

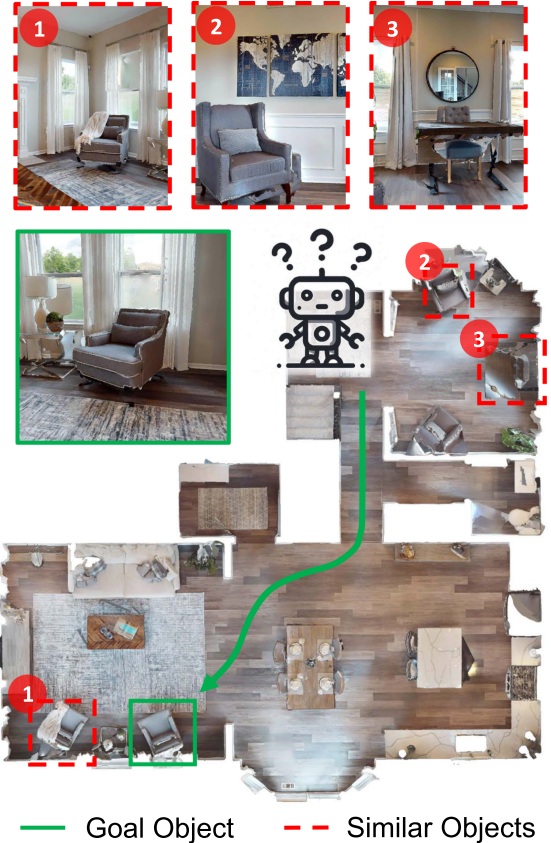


Fig. 1. Illustration of Instance ImageGoal Navigation (IIN), which requires agent to navigate to the object instance depicted in the goal image, while distinguishing it from other visually similar instances.

an agent is to give it natural language instructions, such as “check if my laptop is on the chair”. However, this setting becomes confusing when there are multiple chairs in the house. To overcome this challenge, Krantz et al. [19] propose Instance ImageGoal Navigation (IIN) [20], [21], [22]. In IIN, an agent is presented with an image of a specific object, and its goal is to navigate to the specific object within the least time budget. The goal image is not expected to match the sensor specification or embodiment of the navigating agent, as described in Fig. 1. To accomplish the task, the agent needs to distinguish the target object from different angle of views and ignore potential distractors. This is a challenging task as it involves semantic reasoning, geometry understanding and instance-aware matching.

To address the above issue, previous methods [20], [22], [23], [24], [25], [26], [27] introduce 2D Semantic Bird-Eye-View (BEV) map to tackle this problem. These well-designed explicit map representations are memory-efficient, storing essential

information such as 2D geometry and semantics, and can be directly utilized to calculate the agent's subsequent actions. However, this simple 2D BEV map representation lacks the capacity to retain 3D geometrical information about the environment, rendering it ineffective for navigating cross-floor scenarios. Additionally, BEV maps are unable to preserve the instance-aware features in a scene, which can be crucial for distinguishing between multiple objects of the same class or for tasks requiring fine-grained object interaction.

To avoid the limitation of BEV maps, we propose a new Gaussian Splatting Navigation framework, *i.e.*, GaussNav, for IIN task. Our GaussNav is inspired by the recent advancements in 3D Vision technologies, including Neural Radiance Fields (NeRF) [28] and 3D Gaussian Splatting (3DGS) [29]. These technologies have demonstrated substantial progress in novel view synthesis (NVS) and 3D scene understanding. Although these technologies were not initially developed for navigation tasks, they offer considerable potential for application in this domain. To this end, we develop the Semantic Gaussian map representation, which integrates the representation of geometry, semantics and instance-aware features, and can be directly used for visual navigation.

Our GaussNav framework consists of three stages, including Frontier Exploration, Semantic Gaussian Construction and Gaussian Navigation. First, the agent employs Frontier Exploration to collect observations of the unknown environment. Second, the collected observations are used to construct Semantic Gaussian. By leveraging semantic segmentation algorithms [30], [31], we assign semantic labels to each Gaussian. We then cluster Gaussians with their semantic labels and 3D positions, segmenting objects in the scene into different instances under various semantic categories. This representation is capable of preserving not only the 3D geometry of the scene and the semantic labels of each Gaussian, but also the texture details of the scene, thereby enabling NVS. Third, we render descriptive images for object instances, matching them with the goal image to effectively locate the target object. Upon determining the predicted goal object's position, we can efficiently transform our Semantic Gaussian into grid map and employ path planning algorithms to accomplish the navigation.

To the best of our knowledge, we are the first to introduce 3DGS [29] to embodied visual navigation. In this work, we unify the map representation of geometry, semantics and instance-aware features for visual navigation. Our framework can directly ground the target object with a single goal image input and guide the agent towards it without any additional exploration or verification [22]. Our framework designs are beneficial for effective and efficient visual navigation. We evaluate our method's on both efficacy and efficiency and establish new state-of-the-art records on the challenging Habitat-Matterport 3D dataset (HM3D) [14].

II. RELATED WORK

We briefly discuss related work on differentiable rendering, followed by a broad overview of embodied visual navigation, and finally, we focus on the work most relevant to us: Instance ImageGoal Navigation (IIN).

Differentiable Rendering: To achieve photo-realistic scene capture, differentiable volumetric rendering has gained prominence with the introduction of Neural Radiance Fields (NeRF) [28]. NeRF utilizes a single Multi-Layer Perceptron (MLP) to represent a scene, performing volume rendering by marching along pixel rays and querying the MLP for opacity and color. Due to the inherent differentiability of volume rendering, the MLP representation is optimized to minimize rendering loss using multi-view information, resulting in high-quality novel view synthesis (NVS). The primary limitation of NeRF is its slow training speed. Recent advancements have addressed this issue by incorporating explicit volume structures, such as multi-resolution voxel grids [32], [33], [34] and hash functions [35], to enhance performance.

In contrast to NeRF, 3DGS [29] employs differentiable rasterization. Unlike ray marching, which iterates along pixel rays, 3DGS iterates over the primitives to be rasterized, similar to conventional graphics rasterization. By leveraging the natural sparsity of a 3D scene, 3DGS achieves an expressive representation capable of capturing high-fidelity 3D scenes while offering significantly faster rendering. Comprehensive review of the developments in 3D Gaussian Splatting [36] highlights the method's versatility and applications across various domains. Leveraging these advantages, a growing body of research begins to explore various innovations, including deformable or dynamic Gaussians [37], [38], [39], advancements in mesh extraction and physics simulation [40], [41], [42], as well as applications in Simultaneous Localization and Mapping (SLAM) [43], [44], [45]. This surge of interest in the capabilities of 3DGS leads us to consider its potential for embodied visual navigation. Given that the Gaussian representation inherently encapsulates explicit scene geometry and the parameters required for rendering, we posit that 3DGS can enhance decision-making in map-based visual navigation. The explicit nature of the Gaussian representation provides a rich, condensed form of environmental data, which can be effectively utilized to inform and guide autonomous agents in navigation.

Embodied Visual Navigation: Embodied Visual navigation includes several topics: ObjectGoal Navigation (ObjectNav), Multi ObjectGoal Navigation (MultiON), ImageGoal Navigation (ImageNav), and Instance ImageGoal Navigation (IIN). ObjectNav [24], [46], [47], [48], [49] requires an agent to navigate to any instance of a specified object category within the environment. MultiON [50], [51], [52], [53], on the other hand, tasks the agent to sequentially navigate to a series of objects. ImageNav [26], [54], [55], [56], [57], [58], [59], [60], [61] involves navigating to the camera pose from which a target image is captured. In contrast, IIN [20], [21], [22] requires navigating to the specific instance captured by the camera in the target image. Collectively, these navigation tasks span a spectrum from semantic-level navigation in ObjectNav and MultiON to fine-grained instance-level navigation in ImageNav and IIN, comprehensively capturing the problem space of embodied visual navigation.

Numerous approaches to solving embodied visual navigation utilize deep reinforcement learning (DRL) to develop

end-to-end policies that map egocentric vision to action [26], [46], [56], [62]. However, acquiring skills related to visual scene understanding, semantic exploration, and long-term memory are challenging in an end-to-end framework. Consequently, these methods often incorporate a combination of careful reward shaping [54], pre-training routines [56], and advanced memory modules [59], [63], [64]. In contrast to end-to-end DRL, alternative approaches decompose the problem into sub-tasks that can be optimized in a supervised manner. These sub-tasks include graph prediction via topological SLAM [58], graph-based distance learning [65], [66], and camera pose estimation for last-mile navigation [61]. Chaplot et al. [24] decompose the embodied visual navigation task into exploration, object detection, and local navigation. Building on this, CLIP on Wheels (CoW) [67] utilizes a similar decomposition strategy, focusing on exploration and object localization to handle an open-set object vocabulary. Modular approaches also show promise for effective simulation-to-reality transfer (Sim2Real). Gervet et al. [68] conduct a Sim2Real transfer of both modular and end-to-end systems, demonstrating that modularity effectively mitigates the visual Sim2Real gap that impairs the performance of end-to-end policies.

Modular approaches typically represent the environment using a map and utilize this map to acquire the knowledge necessary for navigation within that environment. Bird's-Eye View (BEV) maps [20], [22], [24], [69] are a form of metric map that project the entire scene from an overhead perspective, representing the occupancy status, exploration state, and semantic categories of specific areas. This map representation encodes information about each region into a grid, achieving accurate spatial awareness. In contrast, topological maps [57], [58], [60] focus more on describing the spatial relationships between different areas in a scene. This approach allows agents to navigate based on the connectivity of spaces rather than relying solely on geometric coordinates. To represent a scene more meticulously, 3D-aware maps have been proposed [49], [70], [71], [72]. Incorporating 3D consistency can enhance perception in navigation. Building upon 3D maps, we further advance by adopting 3D Gaussian Splatting (3DGS) [29] as a novel map representation. This enables the map to synthesize appearance views of specific object instances, which we have demonstrated to be effective in the Instance ImageGoal Navigation task.

Instance ImageGoal Navigation: The IIN task introduces distinct challenges compared to the ImageGoal Navigation. First, in IIN, the goal image must depict a specific object instance. In contrast, ImageGoal Navigation may use randomly captured photos that could include insignificant elements, such as large white walls. Second, the camera parameters used to capture the goal image do not necessarily match those of the agent's camera. Therefore, to succeed in IIN, an agent must be adept at identifying the target object among numerous candidates of the same class with goal object and recognizing it from various viewpoints. To address the above challenges, Krantz et al. [20] develop a general pipeline for aligning the same object from different angle of views. Bono et al. [21] present an end-to-end approach in the IIN task, while Lei et al. [22] propose a method that mimics human behaviour for verifying objects at a distance.

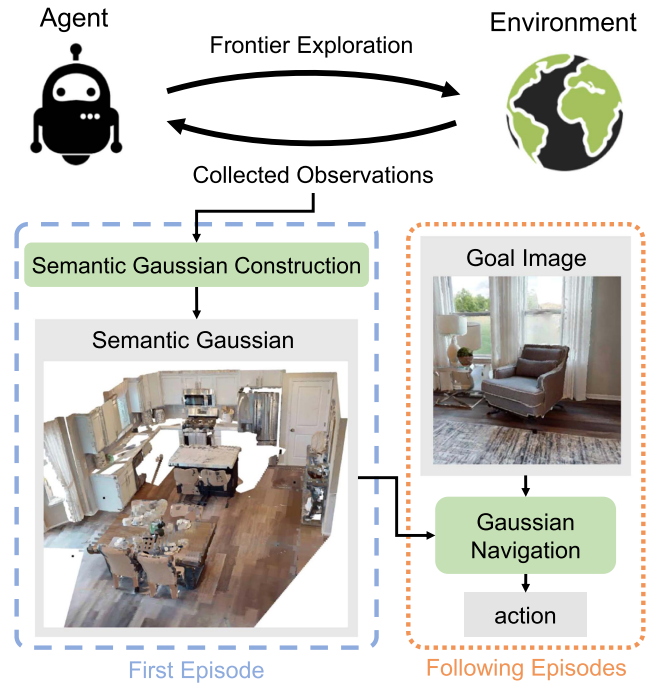


Fig. 2. Framework Overview. In the first episode of a scene, the agent uses frontier exploration to gather observations of the unknown environment, constructing a Semantic Gaussian. In subsequent episodes, the pre-constructed Semantic Gaussian is utilized by Gaussian Navigation to ground the goal object and guide the agent towards it.

Existing methods focus partly on designing sophisticated modules [20], [22] and partly on pre-training on pretext tasks [21]. Our approach differs in that we concentrate on designing a new map representation. Through this novel form of map, we can better establish the connection between target description and target locations, thereby facilitating visual navigation.

III. METHODS

A. Overview

In the IIN task, at the start of a new episode e , the agent is given a goal image I_g that features a specified object instance O_g . The agent's goal is to navigate to the referred instance O_g within the environment. At each timestep t , the agent acquires observations which include an RGB image I_t , a depth image D_t , and sensor pose reading P_t . Utilizing this information, the agent must decide upon and execute an action a_t . When calling stop action, the episode is considered successful only if the agent is within a certain range of the goal object.

To accomplish the IIN task, we propose a modular framework called Gaussian Splatting for Visual Navigation (GaussNav), as depicted in Fig. 2. In a new environment, the Instance ImageGoal Navigation in our proposed framework consists of three stages: Frontier Exploration, Semantic Gaussian Construction, as depicted in Fig. 4, and Gaussian Navigation, illustrated in Fig. 5. Initially, during the first episode within an unknown environment, the agent employs frontier exploration to explore the environment and collect observations. We then use our proposed Semantic Gaussian to reconstruct the scene. Subsequently, in the

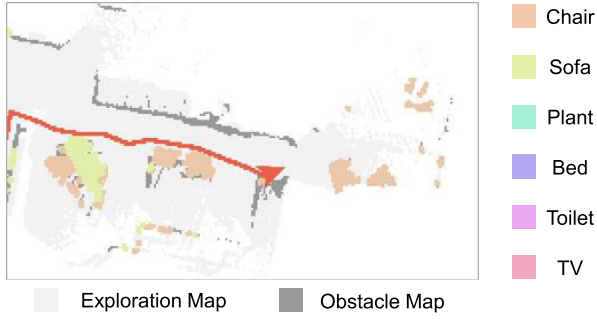
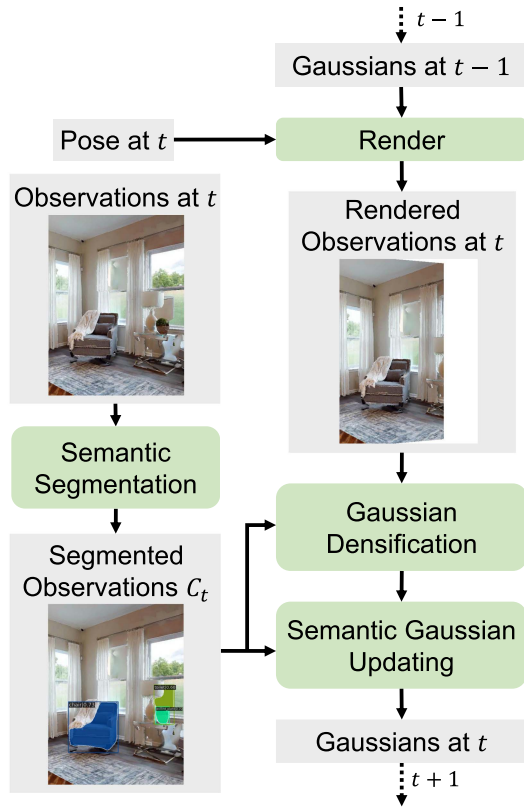


Fig. 3. Exploration Map and Obstacle Map.

Fig. 4. An illustration of Semantic Gaussian Construction. At timestep t , the pipeline updates the Gaussians from $t - 1$ through densification and updating, which involves a comparison between the rendered RGB and depth images against the current input training views. Concurrently, semantic labels are assigned to the densified Gaussians using the segmented images. Finally, the Gaussians are refined through differentiable rendering.

following episodes, the agent leverages the Semantic Gaussian to ground the object instance o depicted in the goal image and navigate to it. This process effectively transforms the IIN task into a more manageable PointGoal Navigation task.

B. Frontier Exploration

In the first episode of an unexplored environment, the agent simultaneously maintains two types of maps, an exploration map and an obstacle map, as illustrated in Fig. 3. The exploration map delineates the regions of the environment that have been explored, while the obstacle map marks the obstacles in the scene. By detecting the contours of the exploration map

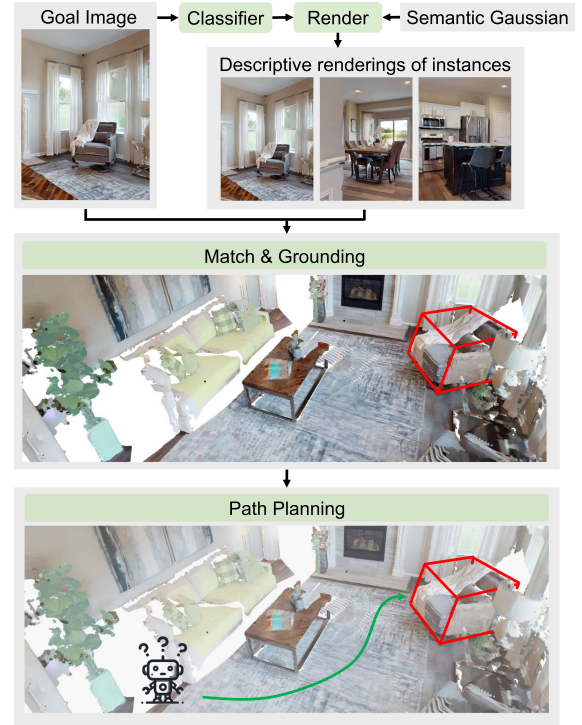


Fig. 5. An illustration of Gaussian Navigation. Our approach begins with the classification of the goal image using pre-constructed Semantic Gaussian. Upon determining the predicted class, we generate descriptive images around instances belonging to that class. These images are then matched with the target object to identify and ground the goal instance. Utilizing the map and the established goal, the agent employs path planning to compute the sequence of actions.

and excluding areas in the obstacle map, the agent sets the closest frontier point as a waypoint to facilitate exploration. The frontier-based exploration strategy is a well-established approach in robotics and autonomous navigation [73], [74]. It involves identifying the boundaries between the explored and unexplored regions of the environment, known as frontiers. The agent then selects the nearest accessible frontier point as its next target for exploration. This decision is based on the distance to the frontier, the navigability of the path, and the potential information gain from exploring that frontier [75]. By iteratively exploring the nearest frontiers, the agent efficiently covers the entire environment while avoiding obstacles and previously explored areas. Through the application of this frontier-based exploration strategy, the agent collects observations of the entire environment.

C. Semantic Gaussian Construction

Our Semantic Gaussian is represented by a group of Gaussians, with each Gaussian characterized by a minimal set of nine parameters: a triplet for the RGB color vector $\mathbf{c}$, a triplet delineating the centroid $\boldsymbol{\mu} \in \mathbb{R}^3$, a scalar representing the radius r , a scalar quantifying the opacity $\alpha \in [0, 1]$, and a scalar representing the category label l . Different from original 3DGS [29], we simplify the representation of Gaussian by using only view-independent color and constraining the Gaussian to be isotropic. This simplification enhances computational efficiency while

reducing memory requirements. For a comprehensive understanding of 3DGS [29], we highly recommend readers consulting the original paper [29]. Our Semantic Gaussian Construction can be described as an iterative process comprising two alternating steps: Gaussian Densification and Semantic Gaussian Updating, as depicted in Fig. 4. Gaussian Densification initializes new Gaussians in the Semantic Gaussian at each new incoming frame, while Semantic Gaussian Updating refines the parameters of each Gaussian through Differentiable Rendering.

Differentiable Rendering: 3DGS [29] renders an RGB image as follows: given a collection of 3D Gaussians and camera pose, first sort all Gaussians from front to back. RGB images can then be efficiently rendered by alpha-compositing the splatted 2D projection of each Gaussian in order in pixel space. The rendered color of pixel $\mathbf{p} = (u, v)$ can be written as:

$$\hat{I}(\mathbf{p}) = \sum_{i=1}^n c_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p})), \quad (1)$$

where $f_i(\mathbf{p})$ is computed as follows:

$$f(\mathbf{x}) = o \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}\|^2}{2r^2}\right). \quad (2)$$

The $\boldsymbol{\mu}$ and r are the splatted 2D Gaussians in pixel-space:

$$\boldsymbol{\mu}^{2D} = K \frac{E_t \boldsymbol{\mu}}{d}, \quad r^{2D} = \frac{f r}{d}, \quad \text{where } d = (E_t \boldsymbol{\mu})_z. \quad (3)$$

Here, K is the camera's intrinsic matrix, E_t embodies the extrinsic matrix that encodes the camera's rotation and translation at time t , f denotes the known focal length, and d is the depth of the i^{th} Gaussian relative to the camera's coordinate frame.

Render: Different from 3DGS [29], we differentially render depth and silhouette image, which determines the visibility and will be used for the next Gaussian Densification and Updating. The depth D and silhouette image S at pixel $\mathbf{p}$ is rendered as follows:

$$\hat{D}(\mathbf{p}) = \sum_{i=1}^n d_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p})), \quad (4)$$

$$\hat{S}(\mathbf{p}) = \sum_{i=1}^n f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p})). \quad (5)$$

Above these, we also render the semantic segmentation results as follows:

$$\hat{C}(\mathbf{p}) = \sum_{i=1}^n l_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p})). \quad (6)$$

Semantic Segmentation: As depicted in Fig. 4, the agent's RGB observation I_t is segmented into C_t using Mask-RCNN [30]. The segmented result will be used for initializing new Gaussians in Gaussian Densification and supervising the Gaussians' parameters in Semantic Gaussian Updating.

Gaussian Densification: Gaussian Densification is performed by comparing the rendered results at position P_t using Gaussians from $t - 1$ with the ground truth. This process adds new Gaussians where the previous Gaussians fail to represent the scene in

the new observations. Following Keetha et al. [43], we add new Gaussians based on a densification mask to determine which pixels should be densified:

$$M(\mathbf{p}) = \left(\hat{S}(\mathbf{p}) < 0.5\right) + \left(D(\mathbf{p}) < \hat{D}(\mathbf{p})\right) \left(L_1(\hat{D}(\mathbf{p})) > 50\text{MDE}\right), \quad (7)$$

where the first term represents where the Semantic Gaussian is not adequately dense, and the second term indicates where the ground-truth depth is in front of the predicted depth and the depth error is greater than 50 times the median depth error (MDE).

Semantic Gaussian Updating: After densifying current Semantic Gaussian, we update the parameters of Gaussians given poses and observations. This is done by differentiable-rendering and gradient-based-optimization, which is equivalent to the "classic" problem of fitting a radiance field to images with known poses. Specifically, we update the parameters of Gaussians by minimizing the RGB, depth and segmentation errors.

D. Gaussian Navigation

To navigate using a constructed Semantic Gaussian, we propose Gaussian Navigation, as illustrated in Fig. 5. We first classify the goal image I_g to predict the semantic label $\hat{l}_g$ of the current target, such as 'chair' in Fig. 5. This semantic label is then used to query the relevant Gaussians. For each relevant object instance of the same class, we render descriptive images. These renderings are compared with the goal image to ground the goal object, predicting its position $\hat{P}_g$. Finally, the Path Planning module generates a feasible path and determines the agent's action.

Classifier: In IIN task, the agent receives an image depicting the target object I_g . However, comparing this image with renderings from navigable points within the entire scene becomes exceedingly time-inefficient due to the vastness of the search space. Consequently, with our Semantic Gaussian G_s , we only need to search for instances of the object category corresponding to the goal object. Therefore, we first classify the goal image I_g into target category label $\hat{l}_g$. We use the goal images on the train split of HM3D-SEM [14] to finetune the image classification model, i.e., ResNet50 [76], pretrained on ImageNet [77].

Match & Grounding: With the predicted target object's label $\hat{l}_g$, we identify all candidate objects that share the same class label. For each candidate instance, we generate a set of descriptive images by rendering the object from multiple viewpoints to capture its features. Specifically, for one training view containing possible candidate objects, we augment it to n_v views by novel view synthesis (NVS). we denote the transformation from the camera to the world coordinate system of the training view as $c2w$, and record the translation of the potential target object in the camera frame as $\mathbf{t}_c^o$. We define the rotation matrix from the object to the world frame using the *forward*, *right*, and *up* vectors. The *forward* direction is the vector from the training view to the potential target object, namely $\mathbf{t}_c^o$. The *up* direction is the vector $[0, -1, 0]$, and the *right* vector is orthogonal to both the *forward* and *up* vectors, forming a right-handed coordinate system with the *right*, *up*, and *forward* vectors. The translation

vector from the object to the world frame is

$$\mathbf{t}_w^o = \mathbf{c2w} \times \mathbf{t}_c^o. \quad (8)$$

Together, the rotation matrix and translation vector constitute the rigid transformation matrix $\mathbf{o2w}$ from the object to the world frame.

We define the rotation matrices around the y -axis and x -axis as $\mathbf{R}_y(\theta)$ and $\mathbf{R}_x(\theta)$, respectively, representing new viewpoints formed by rotating around the object by an angle θ in the horizontal and vertical directions. Thus, the transformation from the camera to the world frame for new viewpoints in the horizontal direction is

$$\mathbf{c2w}_h(\theta) = \mathbf{o2w} \times \mathbf{R}_y(\theta) \times \mathbf{w2o} \times \mathbf{c2w}, \quad (9)$$

and in the vertical direction:

$$\mathbf{c2w}_v(\theta) = \mathbf{o2w} \times \mathbf{R}_x(\theta) \times \mathbf{w2o} \times \mathbf{c2w}. \quad (10)$$

In experiments, when $n_v = 1$, we do not perform NVS; when $n_v = 3$, we perform NVS at $\theta = \pm 15^\circ$ (both horizontal and vertical); and when $n_v = 5$, we use $\theta = \pm 15^\circ, \pm 30^\circ$.

After NVS, the original training views are augmented. The augmented rendering set for the i -th object instance is denoted as S_i . Let n denote the observed instances of the same class as target object, then the universal set of S can be formulated as:

$$S = \{S_1, S_2, \dots, S_n\} \quad \text{for } i = 1, 2, \dots, n. \quad (11)$$

To distinguish the target object from these candidates, we can formulate the question as:

$$i_{\max} = \arg \max_{s \in S_1} \Omega(s), \max_{s \in S_2} \Omega(s), \dots, \max_{s \in S_n} \Omega(s) \quad (12)$$

for $i = 1, 2, \dots, n$,

where $\Omega(\cdot)$ is defined as the matched number of keypoints between renderings and goal image I_g . Specifically, for the rendering $s \in S_i$ of the i -th object instance and the goal image I_g , we extract the pixel-wise (x, y) coordinates of the keypoints and their associated feature descriptors V_t using DISK [78]. That is:

$$(K_t, V_t) = \text{DISK}(s), \quad (K_g, V_g) = \text{DISK}(I_g). \quad (13)$$

Subsequently, the matched pairs $(\hat{K}_t, \hat{K}_g)$ are computed using LightGlue [79]. The feature matching process is formulated as follows:

$$(\hat{K}_t, \hat{K}_g) = \text{LightGlue}((K_t, V_t), (K_g, V_g)). \quad (14)$$

Thus, the number of matched points, *i.e.*, the length of $\hat{K}_t$ or $\hat{K}_g$, is denoted as Ω . The candidate object whose rendered images yield the highest number of matched keypoints is then selected as the target object, as shown in (12).

When the object instance is selected, we ground the object in the Semantic Gaussian. Due to the presence of outliers, which are caused by errors in semantic segmentation, we perform clustering on the instances on the map. Specifically, we use Density-based Spatial Clustering of Applications with Noise

(DBSCAN),¹ which groups together points that are closely packed together, marking as outliers points that lie alone in low-density regions. With the precise selected object instance's location, we can easily transform the IIN task to a PointGoal task.

Path Planning: The Semantic Gaussian is not designed for path planning. We first convert the Semantic Gaussian into a point cloud, whereby each Gaussian is reduced to a single point in the point cloud representation. The point cloud is then voxelized into 3D voxels M_{3D} , then the 3D voxels M_{3D} are projected to 2D BEV grids M_{2D} . Here, we use the 2D projection of Semantic Gaussian rather than the 2D geometric map used in pre-exploration to maintain a consistent representation throughout the pipeline. Our Semantic Gaussian initializes Gaussians using point cloud derived from depth image and does not prune any Gaussian in the optimization stage. Thus, 2D projection of Semantic Gaussian or 2D geometric map are fundamentally equivalent.

Given 2D BEV grid map M_{2D} , along with the agent's starting position and the goal's location, we can efficiently generate a shortest distance field using FMM. Each point within this field encapsulates the minimal distance necessary to traverse from the starting point to the goal. We extract a relevant subset of this distance field that falls within the agent's operational range. Subsequently, a waypoint is chosen from this subset to ensure it avoids any intersections with obstacles while adhering to a local minimum in the distance field. With the selected waypoint, the agent can readily calculate an action based on the angle and distance from its current state. The agent iterates the aforementioned process to generate a sequence of actions, continuing until the destination is reached.

IV. EXPERIMENTS

This section first details our experiment setup, followed by a comparative analysis with state-of-the-art approaches. Subsequently, we present an ablation study to evaluate the efficacy of individual components in our proposed methods. Finally, we assess the computational efficiency of our approach, examine error cases, and evaluate the rendering quality.

A. Experiment Setup

Datasets: We use Habitat-Matterport 3D dataset (HM3D) [14] in the Habitat [16] for our experiments. HM3D consists of scenes which are 3D reconstructions of real-world environments with semantic annotations. These scenes are split into three distinct subsets for training, validation, and testing, consisting of 145/36/35 scenes, respectively. We follow the task setting of Instance ImageGoal Navigation (IIN) proposed by Krantz et al. [19]. The episode dataset has been partitioned into three subsets for training, validation, testing, comprising 7,056 K/1 K/1 K episodes respectively. The object depicted by the goal image belongs to the following six categories: {"chair", "couch", "plant",

¹<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>

“bed”, “toilet”, “television”}. On the validation subset, a total of 795 unique object instances have been observed.

Embodiment: We adopt the embodiment parameters from the Hello Robot Stretch platform.² The simulated agent is modeled as a rigid-body cylinder with zero turning radius, a height of 1.41 m, and a radius of 0.17 m. A forward-facing RGB-D camera is affixed at a height of 1.31 m. At each timestep t , the agent’s observation consists of an egocentric RGB image, depth image, goal image and sensor pose reading. Camera specifications, such as mounting height, look-at angle, and field of view (FOV), differ between the agent’s and the goal’s cameras. Specifically, the agent’s camera resolution is 640×480 , whereas the goal’s camera has a resolution of 512×512 with unfixed height and FOV parameters.

Action Space: We use a discrete action space for navigation, comprising four actions: {STOP, FORWARD, TURN_RIGHT, TURN_LEFT}. The STOP action terminates the current episode, while the FORWARD action advances the agent by 25 cm. Rotational actions occur in place: TURN_RIGHT induces a 25-degree clockwise rotation and TURN_LEFT a 25-degree counter-clockwise rotation.

Evaluation Metrics: Following Krantz et al. [19], we evaluate our model with both success and efficiency. We report Success Rate (Success), Success rate weighted by normalized inverse Path Length (SPL). An episode is deemed successful (Success = 1) if the agent invokes the STOP action within a 1.0m euclidean distance from the goal object. SPL is an efficiency measure defined in [81], is given by:

$$\text{SPL} = \frac{1}{N} \sum_{i=1}^N S_i \cdot \frac{l_i}{\max(p_i, l_i)}, \quad (15)$$

where N is the total number of episodes, S_i is a binary success indicator for episode i , l_i is the shortest path distance from the start position to the goal, and p_i is the path length actually traversed by the agent. A higher SPL value indicates more efficient navigation.

B. Comparison to the State-of-the-Art Methods

We evaluate our proposed model against various baselines and previous state-of-the-art work, as presented in Table I. Unlike conventional IIN methods that generate a map for each episode, GaussNav constructs a map across episodes in a scene. The first block lists results from end-to-end baselines, and the second block includes methods designed for the MultiON task but implemented here for the IIN task. The third block of Table I presents the performance of original state-of-the-art IIN methods, while the fourth block reports their performance when adapted to use scene map.

End-to-end Baselines: We evaluate the performance of two end-to-end methods on the IIN task. *RL Baseline* built a network that observes agent RGB ($\mathcal{V}_{RGB}$), agent depth ($\mathcal{V}_D$), the goal image ($\mathcal{V}_G$), GPS coordinates (x, z), and heading (θ). Visual observations are encoded with separate ResNet-18 [82] encoders and GPS and heading are encoded with 32-dimensional linear

TABLE I
PERFORMANCE COMPARISON OF OUR GAUSSNAV WITH PREVIOUS STATE-OF-THE-ART METHODS ON THE HM3D [14] DATASETS ACROSS TWO DIFFERENT METRICS: SUCCESS AND SPL [81]

Method	Success $\uparrow$	SPL $\uparrow$
RL Baseline [19]	0.083	0.035
OVRL-v2 ImageNav [55]	0.006	0.002
OVRL-v2 IIN [55]	0.248	0.118
FGPrompt [80]	0.099	0.028
MultiON Baseline [51]	0.066	0.045
MultiON Implicit [53]	0.143	0.107
MultiON Camera [50]	0.186	0.142
Mod-IIN [20]	0.561	0.233
IEVE Mask RCNN [22]	0.684	0.241
IEVE InternImage [22]	0.702	0.252
Mod-IIN [20] (Scene Map)	0.563	0.323
IEVE Mask RCNN [22] (Scene Map)	0.683	0.331
IEVE InternImage [22] (Scene Map)	0.705	0.347
GaussNav (ours)	0.725	0.578

The table is divided into four sections. The first section presents the results of end-to-end methods. The second section shows the transfer performance of MultiON-related methods on the IIN task. The third section includes the state-of-the-art methods on the IIN task. Finally, the fourth section aims to provide a fair comparison with GaussNav by replacing the episodic map used in these methods from the third section with a scene-specific map, allowing the agent to retain the map from the previous episode.

layers. Then, these features are concatenated and encoded with a 2-layer LSTM. Finally, an action $a(t)$ is sampled from a categorical distribution. The network was trained from scratch using Proximal Policy Optimization (PPO) [83].

OVRL-v2, proposed by Yadav et al. [55], introduced self-supervised pretraining for visual encoders in ImageGoal Navigation. Originally, OVRL-v2 was trained for the ImageGoal Navigation task. Direct application of OVRL-v2 to IIN task without fine-tuning yields suboptimal performance, as indicated by a Success of 0.006 (row 2 in Table I). This reduced efficacy can be attributed to several factors: the transition between scene datasets from Gibson [13] to HM3D [14], differences in robot embodiment from Locobot³ to Stretch, and a shift in the nature of goal destinations from image sources to image subjects. Fine-tuning OVRL-v2 specifically for IIN task on the HM3D dataset significantly improves outcomes, resulting in a Success of 0.248 (row 3 in Table I).

State-of-the-art Methods in MultiON: The MultiON task shares many similarities with the Scene-specific Map representation we employ in evaluating GaussNav. We re-implement several state-of-the-art methods [50], [51], [53] from the MultiON task on the IIN benchmark. It is worth noting that we directly input the semantic category of the target object to the aforementioned methods. As the navigation requirements of the IIN task are at the instance level rather than the semantic level, this discrepancy in task formulation leaves room for performance improvement.

State-of-the-art Methods in IIN: For fair comparison, We evaluate the performance of previous state-of-the-art methods [20],

²<https://hello-robot.com/stretch-2>

³<http://www.locobot.org/>

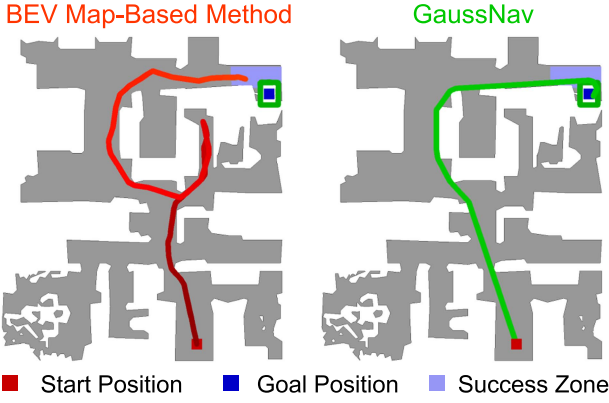


Fig. 6. Trajectory Analysis. Our Semantic Gaussian map representation can allow agent to directly ground target object from a single goal image, thereby facilitating efficient visual navigation.

[22] on the IIN task using two types of map representations: episodic map and scene-specific map. *Mod-IIN* [20] decomposes the IIN task into exploration, goal instance re-identification, goal localization, and local navigation. This method utilizes feature matching to re-identify the goal instance within the egocentric vision and projects the matched features onto a map to localize the goal. Each sub-task is addressed using off-the-shelf components that do not require any fine-tuning.

IEVE [22] employs a modular architecture that dynamically switches between exploration, verification, and exploitation actions. This flexibility empowers the agent to make informed decisions tailored to varying circumstances. *Mod-IIN* and *IEVE* both demonstrate exceptional performance on the IIN task. We also implement the scene-specific map representation in these methods for a fair comparison with GaussNav. As shown in Table I, GaussNav demonstrates a significant performance advantage over all methods. It significantly surpasses all existing models in terms of SPL by a huge margin of 0.231 (last 2 rows in Table I). This result indicates that our Semantic Gaussian map, by preserving the intricate texture details of objects in the scene, enables the agent to directly locate the target object based on the goal image without the need for additional verification.

We attribute the superior performance to the novel map representation of Semantic Gaussian, allowing agent to directly ground goal target without additional exploration or verification, as evidenced by Fig. 6. Unlike previous widely-used BEV map representation, our Semantic Gaussian can allow agent to query Gaussian through semantic label input and render descriptive images of an object instance. Therefore, our GaussNav does not require explicit verification of potential object candidates, unlike BEV map-based approaches. Instead, it selects the most probable candidate from a multitude of possibilities and navigates towards it directly.

C. Ablation Study

To understand the modules of our GaussNav, we consider the following ablations:

GaussNav w.o. Classifier: In Fig. 5, we replace the Classifier’s output with a random generated target category. We observe that

TABLE II
ABLATION STUDY OF GAUSSNAV

Ablations	Success $\uparrow$	SPL $\uparrow$
GaussNav	0.725	0.578
GaussNav w.o. Classifier	0.375	0.291
GaussNav w.o. Match	0.444	0.353
GaussNav w.o. NVS	0.716	0.557
GaussNav w. SIFT	0.655	0.519
GaussNav w. GlueStick [84]	0.723	0.577
GaussNav w. GT Match	0.850	0.672
GaussNav w. GT Goal Localization	0.946	0.744

We study the impact of classifier, match module, novel view synthesis (NVS) and different local feature extraction and matching algorithms on our GaussNav. The last two rows describe the performance of our method using ground truth match results and goal position.

TABLE III
RESULTS OF MATCHING SUCCESS AND TIME TAKEN WITH OR WITHOUT CLASSIFIER

Method	Metric	Ch.	So.	Pl.	Bed	Tol.	TV
w.o. clf	Success $\uparrow$	0.821	0.873	0.798	0.914	0.936	0.829
	Time (s) $\downarrow$	30.9	24.8	33.2	18.1	24.6	31.9
w. clf	Success $\uparrow$	0.782	0.859	0.854	0.878	0.702	0.847
	Time (s) $\downarrow$	15.7	8.09	11.6	5.97	1.85	2.74

Abbreviations: CLF = Classifier, CH. = Chair, SO. = Sofa, PL. = Plant, TOL. = Toilet.

the Success drops to 0.375 and the SPL decreases to 0.291 (row 2 in Table II). To better evaluate the classifier, we design the following experiment. We define the training views from the full space of candidate objects as the complete set, and the training views filtered by the classifier as a subset. Given the goal image and the keypoint matcher, we compute, in each set respectively, the view with the largest number of matching keypoints. If this view contains the goal object, we consider it a success; otherwise, a failure. We also record the total time spent in the entire local feature matching process to assess the impact of the classifier on efficiency. Here, we do not use the navigation metrics such as Success and SPL because this allows us to eliminate the influence of irrelevant factors, such as path planning errors. Our experimental results are shown in Table III. As can be seen, the total success without the classifier is slightly higher than that with the classifier (an increase of 0.039), but the improvement is limited. However, the time taken is 2.5 times that with the classifier, making it significantly less efficient. Therefore, considering the trade-off between performance and efficiency, we choose to include the classifier as a component of GaussNav framework.

GaussNav w.o. Match: The Match module is designed to distinguish the target from candidates with the same class. Without the Match module, we randomly select from these candidates. The Success and SPL falls to 0.444 and 0.353 (row 3 in Table II). This can be attributed to the random selection of candidate instances without Match module.

GaussNav w. SIFT or GlueStick [84]: To evaluate the impact of different extractors and matching algorithms on our GaussNav, we replace the combination of DISK [78] + LightGlue [79].

We employ both SIFT + FLANN and GlueStick [84] as alternatives. The first replacement represents a greater decrease, (row 4 in Table II) and the second only displays a slight decrease (row 5 in Table II). These results demonstrate the varying performance of different local feature matching algorithms on the HM3D dataset.

NVS Analysis: We conduct experiments to evaluate the impact of the number of rendered images n_v and the upper bound of NVS. Specifically, we perform ablation studies on NVS with $n_v = 3$ and $n_v = 5$ in both horizontal and vertical directions, and also evaluate the upper bound achievable using GT rendered results. To avoid the influence of other factors in navigation, we consider it a success only if the image retrieved through keypoint matching contains the target object; otherwise, it is considered a failure. The specific evaluation metrics are detailed in **GaussNav w.o. Classifier**. The experimental results are presented in Table V. Overall, utilizing NVS is beneficial for successfully recognizing objects. However, a large n_v does not necessarily yield positive effects (see Table V, vertical $n_v = 5$ versus vertical $n_v = 3$). This is because our Semantic Gaussian map may have only a few observational viewpoints for a particular object, unlike the dozens available in traditional 3DGS. Therefore, when rendering from novel viewpoints, artifacts such as holes may occur, reducing the number of features, as visualized in Fig. 8. Nevertheless, for GT NVS, the improvement is significant.

To further study the impact of NVS without pre-exploration, we adopt the same framework as IEVE [22], with the only difference being the map representation. IEVE uses 2D BEV map while GaussNav implements Semantic Gaussian map. Additionally, GaussNav performs NVS whenever a similar object appears in each observation. The results are presented in Table VI. Our Semantic Gaussian map enhances the existing observations without relying on pre-exploration. While the improvement may not be as prominent as the ability of instance-level object localization, the result of last row in Table VI indicates promising direction for future work.

D. Efficiency Analysis

We analyze the temporal efficiency of our approach from two perspectives: preprocessing and agent-environment interaction. In the context of our method, “preprocessing” refers to the process of matching and grounding target objects given the Semantic Gaussian. The latter perspective compares the runtime frame rate of our method with various other approaches.

Efficiently locating the target object within the map is crucial for our method, which leverages renderings to match and ground the goal object. To reduce the computational cost of comparing all possible observations at each navigable viewpoint, we introduce Semantic Gaussians. This technique groups object instances under their corresponding semantic labels. This optimization significantly reduces the search space by limiting comparisons to several descriptive renderings of each instance. For example, in the scene CrMo8WxCyVb, the navigable area is quantified to be 54.03 m^2 . This space can be discretized into approximately 54 squares, each with an area of 1 m^2 . Positioned within each square, the agent can observe its surroundings from

TABLE IV
NUMBER OF OBJECT INSTANCES ACROSS DIFFERENT CATEGORIES IN THE FIRST FLOOR OF SCENE CrMo8WxCyVb

Chair	Sofa	TV	Plant	Toilet	Bed
11	2	1	3	1	0

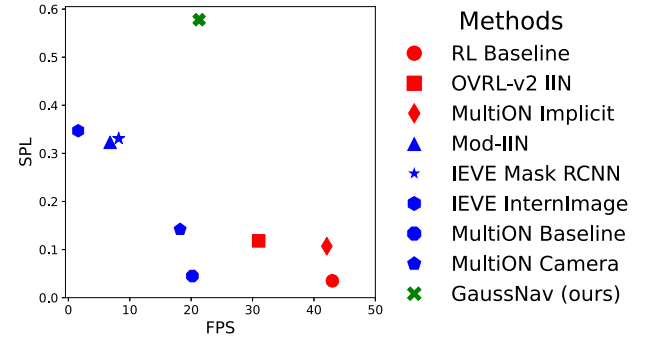


Fig. 7. SPL and FPS Analysis. Red markers represent end-to-end methods, while blue markers indicate modular approaches. Our GaussNav belongs to the modular approach, achieving the highest frame rate among modular methods while attaining the highest SPL across all approaches in IIN task.

12 distinct viewing angles, covering a full 360 deg with each angle spanning 30 deg. Thus, the original search space for this single floor would consist of $12 \times 54 = 648$ potential observations. By applying our grouping strategy based on semantic labeling, the search space is considerably narrowed. We count the number of different categories of object instances within the CrMo8WxCyVb scene, as demonstrated in Table IV. To locate a “chair” — assuming we render each object instance from three unique viewpoints — the resulting search space is reduced to merely $3 \times 11 = 33$ observations. We render three observations of a single instance to ensure rendering quality, which is achievable only when the new viewpoint largely overlaps with the training views. This optimization yields a significant improvement in time efficiency.

We also compare the runtime frame rates of different methods in a single Habitat environment using an NVIDIA GeForce RTX 3090 GPU and 10 CPU cores. As shown in Fig. 7, our method maintains a high efficiency of over 20 FPS while achieving the highest SPL among the compared approaches. To achieve higher runtime speed, our GaussNav projects the Semantic Gaussian to obtain a 2D grid map. By utilizing the predicted target location from the “preprocessing”, we then employ Fast Marching Method (FMM) for path planning. The reason why our method outperforms various Modular Methods in terms of speed is that we do not rely on additional modules such as semantic segmentation [20], [22], local feature matching [20], [22], or switch module [22] during the navigation. This simplification enables GaussNav to operate more efficiently. We also provide a qualitative example of our GaussNav navigating to the target object instance, as depicted in Fig. 9.

E. Error Analysis

The performance of the proposed model is still far from perfect. We would like to understand the error modes for future

TABLE V
RESULTS OF THE NUMBER OF RENDERED IMAGES n , DIFFERENT DIRECTIONS (VERTICAL OR HORIZONTAL) AND WHETHER TO USE GROUND TRUTH RENDERING'S IMPACT ON THE MATCHING SUCCESS

Metric	original	GT original	horizontal		vertical		GT horizontal		GT vertical	
	$n_v = 1$	$n_v = 1$	$n_v = 3$	$n_v = 5$	$n_v = 3$	$n_v = 5$	$n_v = 3$	$n_v = 5$	$n_v = 3$	$n_v = 5$
Success $\uparrow$	0.811	0.815	0.824	0.827	0.832	0.831	0.835	0.846	0.839	0.845
Time (s) $\downarrow$	11.5	11.7	32.1	53.9	33.1	54.2	32.7	54.1	33.4	55.1

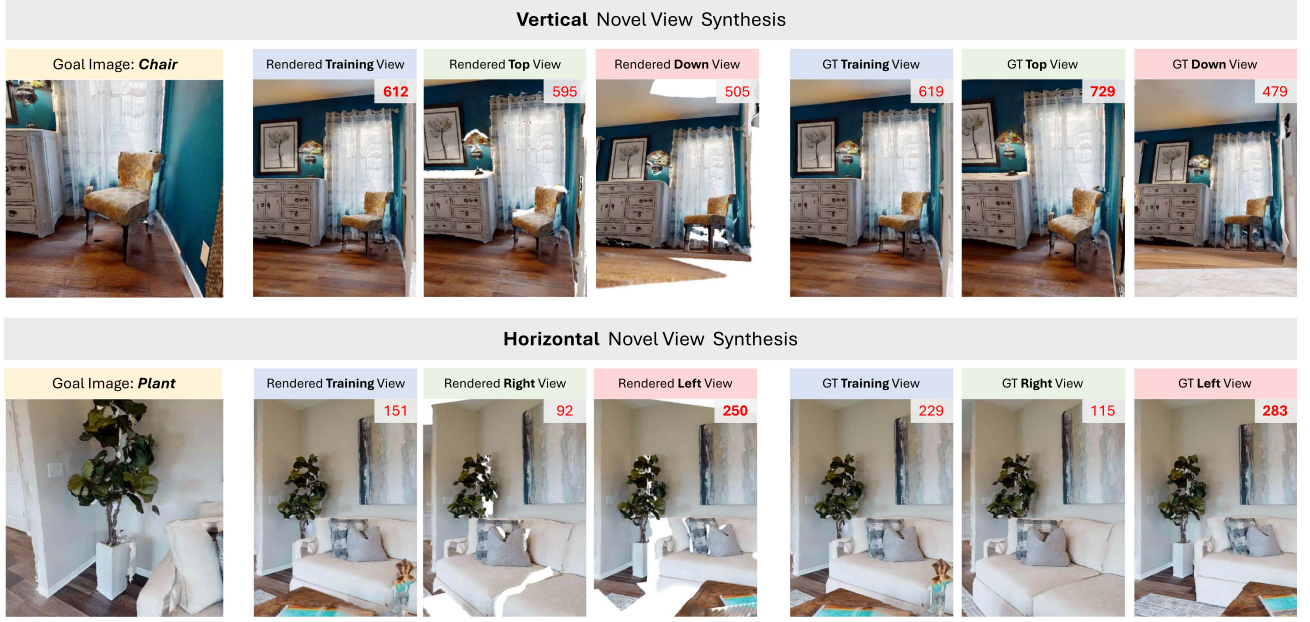


Fig. 8. Visualization of novel view synthesis results. The visualization results include horizontal and vertical novel view synthesis results with $\theta = \pm 15^\circ$. The upper right corner of each image shows the number of matched keypoints with the goal image.

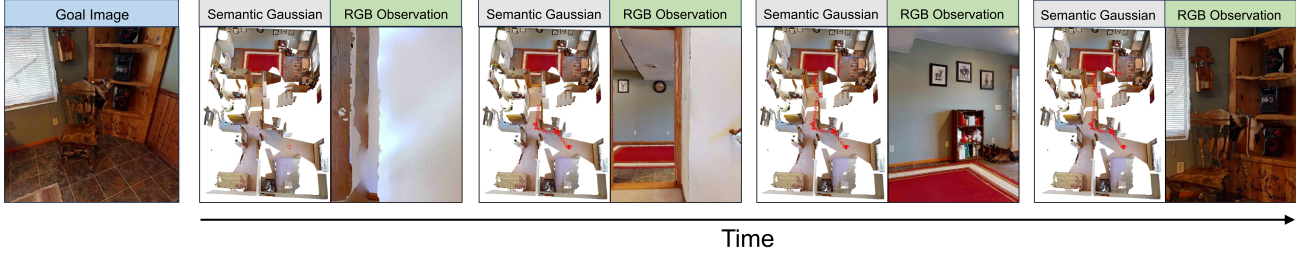


Fig. 9. Qualitative example of our GaussNav agent performing IIN task in the Habitat simulator. When randomly initialized in the environment, the agent is given a goal image depicted a target object. Gaussian Navigation directly grounds the target object in the Semantic Gaussian and guide the agent towards it.

TABLE VI
PERFORMANCE COMPARISON OF IEVE [22] AND GAUSSNAV

Method	Ver. NVS	Hor. NVS	GT	Success $\uparrow$	SPL $\uparrow$
IEVE [22]	-	-	-	0.702	0.252
GaussNav (ours)	$\checkmark$	$\times$	$\times$	0.713	0.259
GaussNav (ours)	$\times$	$\checkmark$	$\times$	0.715	0.261
GaussNav (ours)	$\checkmark$	$\checkmark$	$\times$	0.723	0.265
GaussNav (ours)	$\checkmark$	$\checkmark$	$\checkmark$	0.747	0.289

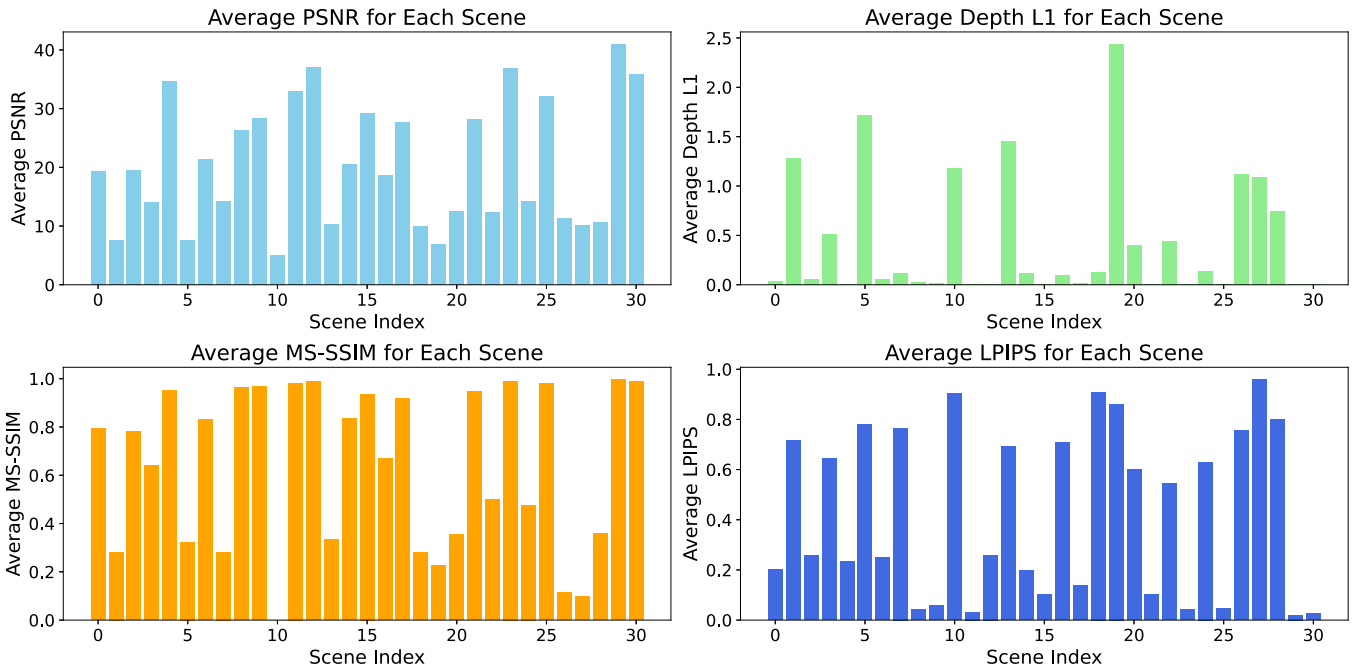
We compare GaussNav with the state-of-the-art method IEVE using different map representations. (abbreviations: Ver. = vertical, Hor. = horizontal).

improvements. Our analysis identifies two sources of error: the first being the model's inability to consistently match the target from instance renderings, and the second, inaccuracies in goal localization. To quantify the impact of these error sources,

we conduct an evaluation of our model using a ground truth Match module and an accurate goal localization. The first one means agent can correctly recognize the target from candidate observations, and the second suggests agent is directly provided with the ground truth goal position. The variant equipped with a ground truth Match module (**GaussNav w. GT Match** in Table II) shows that Success can be enhanced by approximately 0.127 (rows 5 and 6 in Table II). Furthermore, when the model is augmented with both ground truth Match and Goal Localization, denoted as **GaussNav w. GT Goal Localization**, we observe an increase in Success from 0.850 to 0.946, as indicated in rows 6 and 7 in Table II. Improvements in the first error source may be achievable through the development of a more robust re-identification algorithm. As for the second source, a more



Fig. 10. Semantic Gaussian visualization results.

Fig. 11. Rendering quality of our Semantic Gaussian Construction results on the HM3D validation dataset. The x -axis indicates different scene indices with the corresponding floor height.

precise Grounding strategy could yield better results. These insights not only highlight the model's current shortcomings but also chart a course for subsequent refinement efforts.

F. Gaussian Construction Results

As illustrated in Fig. 10, we provide visualization results of our Semantic Gaussian. These examples demonstrate the effectiveness of our Semantic Gaussian representation across a diverse range of scenarios. By presenting a more extensive collection of results, we aim to showcase the robustness and applicability of our approach in handling various scene complexities and object compositions.

In Fig. 11, we present an quantitative evaluation of the rendering quality produced by our Semantic Gaussian Construction

method on the HM3D validation dataset [14]. To align with the constraints of IIN task [19], we divide each scene within HM3D into separate floors and restrict the agent's movement to within a single floor, as the IIN task [19] inherently ensures that both the agent's starting location and the target's position are on the same floor.

To quantitatively analyze the results in Fig. 11, we observe that the rendering results exhibit a bifurcated trend. For instance, in scenes with indices 29 and 30, the rendered images achieve a high PSNR of up to 40 and a near-zero depth rendering error. However, the rendering performance for scene 10 is suboptimal. We hypothesize that this polarized rendering quality across different scenes can be attributed to the discrepancy between the simulation and reality. This is evident in Fig. 12, where some renderings from the HM3D dataset using Habitat simulator

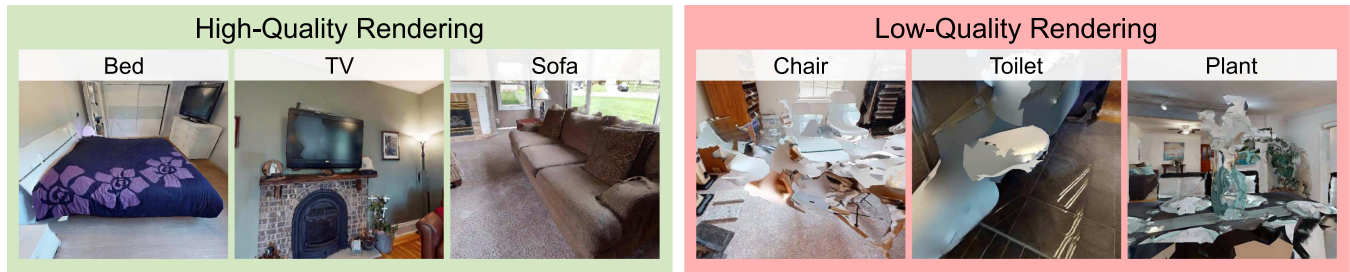


Fig. 12. Observations rendered from HM3D [14] scene dataset using the Habitat [16] simulator.

exhibit low fidelity, particularly in highly textured environments. High-quality reconstruction in such intricate settings is difficult, and utilizing suboptimal renderings as a basis for 3D environment reconstruction can further degrade the quality of the final output. In light of this, for scenes that are poorly reconstructed, we maintain consistency by using the original training views, rather than attempting to render novel views which would likely result in a diminished quality.

V. CONCLUSION

In this work, we introduce a modular approach for visual navigation, *i.e.*, Gaussian Splatting for Visual Navigation (GaussNav). Previous map-based methods largely focus on building 2D BEV map, which cannot represent the 3D geometry and detailed features in a scene. To this end, we propose a novel map representation, Semantic Gaussian, which is capable of preserving the scene's 3D geometry, semantic labels associated with each Gaussian, and intricate texture details. Leveraging this novel representation of map, we directly predict the position of target object depicted in the goal image, thereby transforming IIN into a more tractable PointGoal Navigation task. Our proposed framework achieves state-of-the-art performance, significantly enhancing SPL from 0.347 to 0.578. Furthermore, we analyze the error modes for our model and quantify the scope for improvement along two important dimensions (match and object grounding) in the future work.

REFERENCES

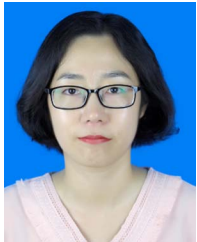
- [1] W. Cheng, X. Dong, S. Khan, and J. Shen, "Learning disentanglement with decoupled labels for vision-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 309–329.
- [2] J. Krantz and S. Lee, "Sim-2-sim transfer for vision-and-language navigation in continuous environments," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 588–603.
- [3] S. Zhang, W. Li, X. Song, Y. Bai, and S. Jiang, "Generative meta-adversarial network for unseen object navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 301–320.
- [4] C. Lin, Y. Jiang, J. Cai, L. Qu, G. Haffari, and Z. Yuan, "Multimodal transformer with variable-length memory for vision-and-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 380–397.
- [5] Y. Qiao, Y. Qi, Y. Hong, Z. Yu, P. Wang, and Q. Wu, "HOP+: History-enhanced and order-aware pre-training for vision-and-language navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 7, pp. 8524–8537, Jul. 2023.
- [6] Q. Cai, L. Zhang, Y. Wu, W. Yu, and D. Hu, "A pose-only solution to visual reconstruction and navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 73–86, Jan. 2023.
- [7] B. Lin, Y. Zhu, Y. Long, X. Liang, Q. Ye, and L. Lin, "Adversarial reinforced instruction attacker for robust vision-language navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 7175–7189, Oct. 2022.
- [8] D. An et al., "ETPNav: Evolving topological planning for vision-language navigation in continuous environments," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Apr. 09, 2024, doi: [10.1109/TPAMI.2024.3386695](https://doi.org/10.1109/TPAMI.2024.3386695).
- [9] J. Wu, Y. Zhou, H. Yang, Z. Huang, and C. Lv, "Human-guided reinforcement learning with sim-to-real transfer for autonomous navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 14745–14759, Dec. 2023.
- [10] X. Wang et al., "Vision-language navigation policy learning and adaptation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 12, pp. 4205–4216, Dec. 2021.
- [11] X. Wang, W. Wang, J. Shao, and Y. Yang, "Learning to follow and generate instructions for language-capable navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 3334–3350, May 2024.
- [12] H. Wang, W. Liang, L. V. Gool, and W. Wang, "Towards versatile embodied navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 36858–36874.
- [13] F. Xia, A. R. Zamir, Z. He, A. Sax, J. Malik, and S. Savarese, "Gibson ENV: Real-world perception for embodied agents," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 9068–9079.
- [14] K. Yadav et al., "Habitat-matterport 3D semantics dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 4927–4936.
- [15] A. Chang et al., "Matterport3D: Learning from RGB-D data in indoor environments," 2017, *arXiv:1709.06158*.
- [16] A. Szot et al., "Habitat 2.0: Training home assistants to rearrange their habitat," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 251–266.
- [17] E. Kolve et al., "Ai2-THOR: An interactive 3D environment for visual AI," 2017, *arXiv:1712.05474*.
- [18] M. Savva et al., "Habitat: A platform for embodied AI research," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9339–9347.
- [19] J. Krantz, S. Lee, J. Malik, D. Batra, and D. S. Chaplot, "Instance-specific image goal navigation: Training embodied agents to find object instances," 2022, *arXiv:2211.15876*.
- [20] J. Krantz et al., "Navigating to objects specified by images," 2023, *arXiv:2304.01192*.
- [21] G. Bono, L. Antsfeld, B. Chidlovskii, P. Weinzaepfel, and C. Wolf, "End-to-end (instance)-image goal navigation through correspondence as an emergent phenomenon," 2023, *arXiv:2309.16634*.
- [22] X. Lei, M. Wang, W. Zhou, L. Li, and H. Li, "Instance-aware exploration-verification-exploitation for instance imagegoal navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16329–16339.
- [23] D. S. Chaplot, D. Gandhi, S. Gupta, A. Gupta, and R. Salakhutdinov, "Learning to explore using active neural SLAM," 2020, *arXiv:2004.05155*.
- [24] D. S. Chaplot, D. P. Gandhi, A. Gupta, and R. R. Salakhutdinov, "Object goal navigation using goal-oriented semantic exploration," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4247–4258.
- [25] S. K. Ramakrishnan, D. S. Chaplot, Z. Al-Halah, J. Malik, and K. Grauman, "PONI: Potential functions for objectgoal navigation with interaction-free learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 18 890–18 900.
- [26] Z. Al-Halah, S. K. Ramakrishnan, and K. Grauman, "Zero experience required: Plug & play modular transfer learning for semantic visual navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 17 031–17 041.

- [27] B. Ma, X. Yin, D. Wu, H. Shen, X. Ban, and Y. Wang, "End-to-end learning for simultaneously generating decision map and multi-focus image fusion result," *Neurocomputing*, vol. 470, pp. 204–216, 2022.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 99–106.
- [29] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, Jul. 2023.
- [30] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2961–2969.
- [31] W. Wang et al., "InternImage: Exploring large-scale vision foundation models with deformable convolutions. arxiv 2022," 2023, *arXiv: 2211.05778*.
- [32] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5501–5510.
- [33] L. Liu, J. Gu, K. Zaw Lin, T.-S. Chua, and C. Theobalt, "Neural sparse voxel fields," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 15 651–15 663.
- [34] C. Sun, M. Sun, and H.-T. Chen, "Direct voxel grid optimization: Superfast convergence for radiance fields reconstruction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5459–5469.
- [35] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–15, 2022.
- [36] G. Chen and W. Wang, "A survey on 3D Gaussian splatting," 2024, *arXiv:2401.03890*.
- [37] J. Luiten, G. Kopanas, B. Leibe, and D. Ramanan, "Dynamic 3D Gaussians: Tracking by persistent dynamic view synthesis," 2023, *arXiv:2308.09713*.
- [38] G. Wu et al., "4D Gaussian splatting for real-time dynamic scene rendering," 2023, *arXiv:2310.08528*.
- [39] Z. Yang, X. Gao, W. Zhou, S. Jiao, Y. Zhang, and X. Jin, "Deformable 3D Gaussians for high-fidelity monocular dynamic scene reconstruction," 2023, *arXiv:2309.13101*.
- [40] Y. Feng et al., "Gaussian splashing: Dynamic fluid synthesis with Gaussian splatting," 2024, *arXiv:2401.15318*.
- [41] T. Xie et al., "Physgaussian: Physics-integrated 3D Gaussians for generative dynamics," 2023, *arXiv:2311.12198*.
- [42] A. Guédon and V. Lepetit, "Sugar: Surface-aligned Gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering," 2023, *arXiv:2311.12775*.
- [43] N. Keetha et al., "Splatam: Splat, track & map 3D Gaussians for dense RGB-D SLAM," 2023, *arXiv:2312.02126*.
- [44] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting SLAM," 2023, *arXiv:2312.06741*.
- [45] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photorealistic dense SLAM with Gaussian splatting," 2023, *arXiv:2312.10070*.
- [46] A. Majumdar, G. Aggarwal, B. Devnani, J. Hoffman, and D. Batra, "ZSON: Zero-shot object-goal navigation using multimodal goal embeddings," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 32 340–32 352.
- [47] H. Wang, A. G. H. Chen, X. Li, M. Wu, and H. Dong, "Find what you want: Learning demand-conditioned object attribute space for demand-driven navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 16353–16366.
- [48] M. Chang, A. Gupta, and S. Gupta, "Semantic visual navigation by watching youtube videos," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4283–4294.
- [49] D. S. Chaplot, M. Dalal, S. Gupta, J. Malik, and R. R. Salakhutdinov, "Seal: Self-supervised embodied active learning using exploration and 3D consistency," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 13 086–13 098.
- [50] P. Chen et al., "Learning active camera for multi-object navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 28 670–28 682.
- [51] S. Wani, S. Patel, U. Jain, A. Chang, and M. Savva, "Multion: Benchmarking semantic map memory using multi-object navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 9700–9712.
- [52] F. Schmalstieg, D. Honerkamp, T. Welschhold, and A. Valada, "Learning long-horizon robot exploration strategies for multi-object search in continuous action spaces," in *Proc. Int. Symp. Robot. Res.*, Springer, 2022, pp. 52–66.
- [53] P. Marza, L. Matignon, O. Simonin, and C. Wolf, "Multi-object navigation with dynamically learned neural implicit representations," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 11 004–11 015.
- [54] Y. Choi and S. Oh, "Image-goal navigation via keypoint-based reinforcement learning," in *Proc. Int. Conf. Ubiquitous Robots*, 2021, pp. 18–21.
- [55] K. Yadav et al., "OVRL-V2: A simple state-of-art baseline for ImageNav and ObjectNav," 2023, *arXiv:2303.07798*.
- [56] K. Yadav et al., "Offline visual representation learning for embodied navigation," in *Proc. Workshop Reincarnating Reinforcement Learn. (RRL) Int. Conf. Learn. Representations (ICLR)*, 2023.
- [57] O. Kwon, J. Park, and S. Oh, "Renderable neural radiance map for visual navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 9099–9108.
- [58] D. S. Chaplot, R. Salakhutdinov, A. Gupta, and S. Gupta, "Neural topological SLAM for visual navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12875–12884.
- [59] N. Savinov, A. Dosovitskiy, and V. Koltun, "Semi-parametric topological memory for navigation," 2018, *arXiv:1803.00653*.
- [60] N. Kim, O. Kwon, H. Yoo, Y. Choi, J. Park, and S. Oh, "Topological semantic graph memory for image goal navigation," in *Proc. Conf. Robot Learn.*, 2022, pp. 393–402.
- [61] J. Wasserman, K. Yadav, G. Chowdhary, A. Gupta, and U. Jain, "Last-mile embodied visual navigation," in *Proc. Conf. Robot Learn.*, 2022, pp. 666–678.
- [62] Y. Zhu et al., "Target-driven visual navigation in indoor scenes using deep reinforcement learning," in *Proc. 2017 IEEE Int. Conf. Robot. Automat.*, 2017, pp. 3357–3364.
- [63] L. Mezghan et al., "Memory-augmented reinforcement learning for image-goal navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 3316–3323.
- [64] Y. Wu, Y. Wu, A. Tamar, S. Russell, G. Gkioxari, and Y. Tian, "Bayesian relational memory for semantic visual navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 2769–2779.
- [65] M. Hahn, D. S. Chaplot, S. Tulsiani, M. Mukadam, J. M. Rehg, and A. Gupta, "No RL, no simulation: Learning to navigate without navigating," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 26 661–26 673.
- [66] D. Shah, B. Eysenbach, G. Kahn, N. Rhinehart, and S. Levine, "Rapid exploration for open-world navigation with latent goal models," 2021, *arXiv:2104.05859*.
- [67] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, "Cows on pasture: Baselines and benchmarks for language-driven zero-shot object navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 23 171–23 181.
- [68] T. Gervet, S. Chintala, D. Batra, J. Malik, and D. S. Chaplot, "Navigating to objects in the real world," *Sci. Robot.*, vol. 8, no. 79, 2023, Art. no. eadf6991.
- [69] R. Liu, X. Wang, W. Wang, and Y. Yang, "Bird's-eye-view scene graph for vision-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 10 968–10 980.
- [70] J. Zhang et al., "3D-aware object goal navigation via simultaneous exploration and identification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 6672–6682.
- [71] S. Tan, K. Sima, D. Wang, M. Ge, D. Guo, and H. Liu, "Self-supervised 3D semantic representation learning for vision-and-language navigation," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, May 14, 2024, doi: [10.1109/TNNLS.2024.3395633](https://doi.org/10.1109/TNNLS.2024.3395633).
- [72] R. Liu, W. Wang, and Y. Yang, "Volumetric environment representation for vision-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16 317–16 328.
- [73] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Comput. Intell. Robot. Automat.*, 1997, pp. 146–151.
- [74] D. Holz, N. Basilico, F. Amigoni, and S. Behnke, "Evaluating the efficiency of frontier-based exploration strategies," in *Proc. Int. Symp. Robot. German Conf. Robot.*, 2010, pp. 1–8.
- [75] M. Juliá, A. Gil, and O. Reinoso, "A comparison of path planning strategies for autonomous exploration and mapping of unknown environments," *Auton. Robots*, vol. 33, pp. 427–444, 2012.
- [76] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015, *arXiv:1512.03385*.
- [77] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [78] M. Tyszkiewicz, P. Fua, and E. Trulls, "Disk: Learning local features with policy gradient," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 14 254–14 265.
- [79] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," 2023, *arXiv:2306.13643*.

- [80] X. Sun, P. Chen, J. Fan, J. Chen, T. Li, and M. Tan, "FGPrompt: Fine-grained goal prompting for image-goal navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 12054–12073.
- [81] P. Anderson et al., "On evaluation of embodied navigation agents," 2018, *arXiv:1807.06757*.
- [82] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [83] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [84] R. Pautrat, I. Suárez, Y. Yu, M. Pollefeys, and V. Larsson, "GlueStick: Robust image matching by sticking points and lines together," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 9706–9716.



Xiaohan Lei is currently working toward the PhD degree in information and communication engineering with the Department of Information Science and Technology, from the University of Science and Technology of China. His research interests include embodied visual navigation, robot manipulation and embodied computer vision.



Min Wang received the BE, and PhD degrees in electronic information engineering from the University of Science and Technology of China (USTC), in 2014 and 2019, respectively. She is working with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center. Her current research interests include binary hashing, multimedia information retrieval and computer vision.



Wengang Zhou (Senior Member, IEEE) received the BE degree in electronic information engineering from Wuhan University, China, in 2006, and the PhD degree in electronic engineering and information science from the University of Science and Technology of China (USTC), China, in 2011. From 2011 to 2013, he worked as a postdoc researcher with Computer Science Department, the University of Texas at San Antonio. He is currently a professor with the EEIS Department, USTC. His research interests include multimedia information retrieval, computer vision,

and computer game. In those fields, he has published more than 100 papers in IEEE/ACM Transactions and CCF Tier-A International Conferences. He is the winner of National Science Funds of China (NSFC) for Excellent Young Scientists in 2018, and Chinese Society of Image and Graphics (CSIG) Young Scientist Award in 2024. He is the recipient of the Best Paper Award for ICIMCS 2012. He received the award for the Excellent PhD Supervisor of Chinese Society of Image and Graphics (CSIG) in 2021, and the award for the Excellent Ph.D Supervisor of Chinese Academy of Sciences (CAS) in 2022. He won the First Class Wu-Wenjun Award for Progress in Artificial Intelligence Technology in 2021. He served as the publication chair of IEEE ICME 2021 and won 2021 ICME Outstanding Service Award. He is currently an associate editor and a lead guest editor of *IEEE Transactions on Multimedia*, and is the recipient of 2023 IEEE Transactions on Multimedia (TMM) Excellent Editor Award.



Houqiang Li (Fellow, IEEE) received the BS, MEng, and PhD degrees in electronic engineering from the University of Science and Technology of China, Hefei, China, in 1992, 1997, and 2000, respectively. He is currently a professor with the Department of Electronic Engineering and Information Science. His research interests include reinforcement learning, multimedia search, image/video analysis, video coding and communication, etc. He has authored and coauthored more than 200 papers in journals and conferences. He is the winner of National Science

Funds (NSFC) for Distinguished Young Scientists, the Distinguished Professor of Changjiang Scholars Program of China, and the Leading Scientist of Ten Thousand Talent Program of China. He is the associate editor (AE) for *IEEE Transactions on Multimedia* and served as the associate editor for *IEEE Transactions on Circuits and Systems for Video Technology*. He served as the general co-chair of ICME 2021 and the TPC co-chair of VCIP 2010. He was the recipient of National Technological Invention Award of China (second class) in 2019 and the recipient of National Natural Science Award of China (second class) in 2015. He was the recipient of the Best Paper Award for VCIP 2012, the recipient of the Best Paper Award for ICIMCS 2012, and the recipient of the Best Paper Award for ACM MUM in 2011.